

SMART ROUTE OPTIMIZER

Algorithm Visualizer

BY

K.koushik
G.SURESH
SRIVARDAN
PAVAN
SATYA
ADHINAV

AP24122230007
AP24122230018
AP24122230029
AP24122230020
AP24122230023
AP24122230021



SRM UNIVERSITY,AP

APRIL 2026

1. Introduction

SmartRoute is an interactive Route Optimizer that demonstrates three fundamental algorithms applied to the Travelling Salesman Problem (TSP) and shortest-path routing. The project was built as part of the CCC algorithm project submission, showcasing Greedy, Dynamic Programming, and Graph algorithms through both a C++ console application and an interactive web-based visualizer.

The Travelling Salesman Problem asks: given a list of cities and distances between them, what is the shortest possible route that visits each city exactly once and returns to the origin? This is an NP-Hard problem with broad real-world applications.

1.1 Objectives

- Implement Greedy, DP, and Dijkstra algorithms for route optimization
- Compare algorithm performance in terms of cost, time, and complexity
- Visualize the algorithms interactively via a web-based tool
- Provide a working C++ console application with real city data
- Demonstrate understanding of time and space complexity trade-offs

1.2 Project Scope

The project covers route optimization using Indian cities as test data. The web visualizer allows interactive creation of city graphs, real-time animation of each algorithm, and a side-by-side comparison of all three approaches.

2. Algorithms Used

Three algorithms were chosen to represent different algorithmic paradigms — Greedy, Dynamic Programming, and Graph-based shortest path. Together they cover the core topics specified in the CCC project requirements.

Algorithm	Type	Time Complexity	Space	Optimal?
Greedy NN	Greedy	$O(n^2)$	$O(n)$	No (Heuristic)
DP Held-Karp	Dynamic Prog.	$O(n^2 \times 2^n)$	$O(n \times 2^n)$	Yes (Exact)
Dijkstra	Graph / Greedy	$O(n^2)$	$O(n)$	Shortest path

2.1 Greedy Nearest Neighbor

The Greedy Nearest Neighbor algorithm is a heuristic approach to solving the TSP. Starting from a given city, it repeatedly visits the closest unvisited city until all cities are visited, then returns to the start.

How It Works:

- Start at the chosen city, mark it as visited
- Find the closest unvisited city to the current position
- Move to that city, add it to the route
- Repeat until all cities are visited
- Return to the starting city to complete the tour

Pseudocode:

```
function greedy(cities[], start):
    visited = {start}, path = [start]
    while unvisited cities remain:
        best = argmin { dist(current, v) for v not in visited }
        path.append(best), visited.add(best)
    path.append(start) // return to start
    return path
```

Time Complexity: $O(n^2)$ — for n cities, the inner loop runs n times per step.

Space Complexity: $O(n)$ — for visited array and path.

Limitation: Does not guarantee the globally optimal solution. In the worst case, it can be 25% worse than optimal.

2.2 Dynamic Programming — Held-Karp (Exact TSP)

The Held-Karp algorithm uses bitmask Dynamic Programming to solve the TSP exactly. It stores the minimum cost to reach each city for every possible subset of visited cities, avoiding redundant computations through memoization.

How It Works:

- Let $dp[mask][u]$ = minimum cost to visit all cities in 'mask', ending at city u
- Start with $dp[1 \ll start][start] = 0$
- For each state (mask, u), extend to all unvisited cities v
- $dp[mask | (1 \ll v)][v] = \min(dp[mask | (1 \ll v)][v], dp[mask][u] + dist(u, v))$
- Reconstruct the path by backtracking through parent pointers

Pseudocode:

```
dp[1<<start][start] = 0
for each mask in 0..2^n:
    for each u in mask:
        for each v not in mask:
            dp[mask | (1<<v)][v] = min(dp[mask | (1<<v)][v],
                                         dp[mask][u] + dist(u, v))
answer = min over all u of { dp[FULL][u] + dist(u, start) }
```

Time Complexity: $O(n^2 \times 2^n)$ — exponential but guaranteed optimal.

Space Complexity: $O(n \times 2^n)$ — for the DP table.

Best for: $n \leq 15$ –20 cities. For larger n , falls back to the Greedy heuristic.

2.3 Dijkstra's Shortest Path

Dijkstra's algorithm finds the shortest path from a source node to all other nodes in a weighted graph. In this project, it is used to build a tour by visiting cities in order of their shortest distance from the source.

How It Works:

- Assign distance 0 to the source, infinity to all others
- Pick the unvisited city with the smallest known distance
- For each unvisited neighbor, relax the edge: $d[v] = \min(d[v], d[u] + w(u, v))$
- Mark the current city as visited and repeat
- Build a tour by visiting cities in order of $d[]$ values

Pseudocode:

```
dist[start] = 0; dist[all others] = INF
while unvisited cities remain:
    u = city with minimum dist that is unvisited
    mark u as visited
    for each neighbor v of u:
        if dist[u] + weight(u,v) < dist[v]:
            dist[v] = dist[u] + weight(u,v)
```

Time Complexity: $O(n^2)$ — optimal for dense graphs.

Space Complexity: $O(n)$ — for distance and visited arrays.

Note: Dijkstra minimizes path cost from the source but does not minimize the total tour length.

3. Implementation

3.1 Technologies Used

Language (Core)	C++17 — Console application with all 3 algorithms
Language (Visual)	HTML5 + CSS3 + Vanilla JavaScript — Web visualizer
Compiler	g++ with -std=c++17 -O2 flags
Libraries	C++ STL: vector, queue, algorithm, chrono, functional
Platform	Any OS — browser for HTML, terminal for C++

3.2 Project Structure

- index.html — Complete interactive web visualizer (single file, no dependencies)
- main.cpp — C++ console application implementing all 3 algorithms
- README.md — Documentation with algorithm explanations and setup guide

3.3 Key Features

- Interactive graph canvas — click to add cities, drag to reposition
- Step-by-step animated route visualization for all 3 algorithms
- Real-time algorithm comparison table with costs and complexities
- Execution time measurement in milliseconds
- Random Indian city generator for quick testing

- C++ interactive mode — enter custom city coordinates
- Right-click to delete nodes in the web visualizer
- Adjustable animation speed slider

4. Results & Analysis

The following results were obtained by running all three algorithms on 8 Indian cities: Mumbai, Delhi, Chennai, Kolkata, Bengaluru, Hyderabad, Pune, and Ahmedabad. Each city was assigned (x, y) coordinates representing their approximate relative positions.

4.1 Test Data — 8 Indian Cities

No.	City	X Coord	Y Coord	Region
1	Mumbai	100	300	West
2	Delhi	300	100	North
3	Chennai	350	500	South
4	Kolkata	550	200	East
5	Bengaluru	280	450	South
6	Hyderabad	300	380	Central
7	Pune	200	340	West
8	Ahmedabad	150	200	West

4.2 Algorithm Output Comparison

Algorithm	Route Distance	Time (ms)	Complexity	Result
Greedy NN	1296.13 units	0.003 ms	$O(n^2)$	Good
DP Held-Karp	1296.13 units	0.083 ms	$O(n^2 \times 2^n)$	OPTIMAL
Dijkstra	2138.69 units	0.002 ms	$O(n^2)$	Shortest path

4.3 Analysis

Both Greedy NN and DP Held-Karp achieved the same optimal cost of 1296.13 units on this dataset, which confirms that for this particular set of cities the greedy heuristic happened to find the global optimum. In general, Greedy is not guaranteed to be optimal.

Dijkstra produced a higher tour cost of 2138.69 units because it is designed to minimize the distance from a single source to all nodes, not to minimize the total round-trip distance. This illustrates the difference between single-source shortest path and TSP.

The DP algorithm took 0.083 ms compared to 0.003 ms for Greedy, reflecting its higher computational cost. However, it is the only algorithm that guarantees the optimal TSP solution.

5. Conclusion

This project successfully implemented and compared three fundamental algorithms — Greedy Nearest Neighbor, Dynamic Programming (Held-Karp), and Dijkstra's Shortest Path — applied to route optimization over Indian cities.

Key learnings from this project:

- Greedy algorithms are fast ($O(n^2)$) but do not guarantee optimal solutions
- Dynamic Programming guarantees optimality at the cost of exponential complexity $O(n^2 \times 2^n)$
- Dijkstra excels at single-source shortest paths but is not designed for full tour minimization
- Algorithm selection must balance optimality requirements with computational constraints

The interactive web visualizer makes it easy to understand how each algorithm traverses the graph differently, reinforcing the theoretical concepts with visual feedback. The C++ implementation demonstrates the algorithms in a production-ready, performant environment.

5.1 Future Scope

- Add more algorithms: A* Search, Ant Colony Optimization, Christofides
- Extend to real GPS coordinates using mapping APIs
- Add weighted edges with actual road distances
- Implement parallel computation for large city datasets
- Mobile-responsive web interface

6. References

- Cormen, T.H. et al. (2022). Introduction to Algorithms, 4th Edition. MIT Press.
- Held, M. & Karp, R.M. (1962). A Dynamic Programming Approach to Sequencing Problems. Journal of the Society for Industrial and Applied Mathematics.
- Dijkstra, E.W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269-271.
- GeeksForGeeks — Travelling Salesman Problem using Dynamic Programming.
- cppreference.com — C++ Standard Library Reference.